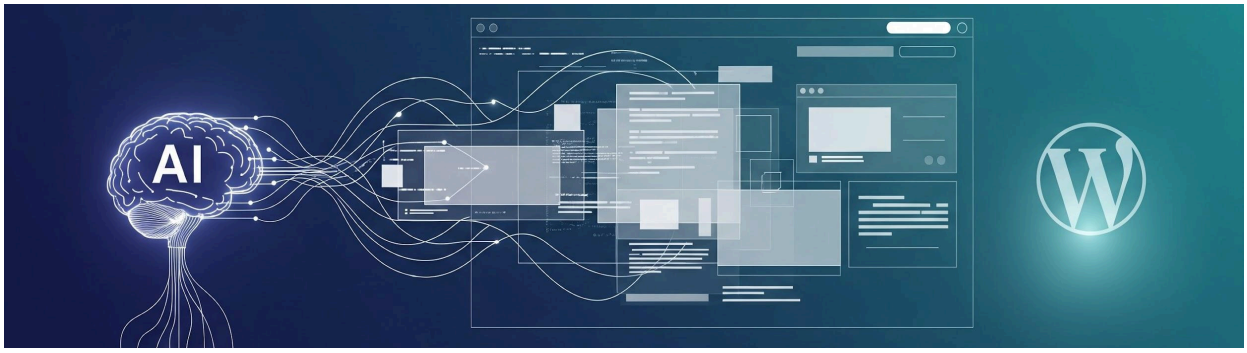

AI-Powered WordPress Block Theme Generator — Challenger Project

Overview

Build a standalone AI assistant that generates complete, structured WordPress Block Themes (using Full Site Editor (FSE) capabilities) based on a user input. The challenge is to go beyond generic, boilerplate themes. The resulting theme must be well-structured, visually appealing, and composed *entirely* of standard, core, and well-known pattern blocks. Crucially, **the generated theme must not use the Custom HTML block for any structural or visual element.**

This project requires thoughtful engineering to translate abstract user intent into concrete, structured JSON/PHP files that define a modern, high-quality WordPress theme.

The Challenge

Create a web application where users can:

1. **Input Theme Criteria**: Provide a natural language description (e.g., "A dark mode blog theme for photographers with a large, centered hero section and sticky navigation.") and define specific technical criteria (e.g., color palette, typography).
2. **Generate Theme**: Request the AI to generate the complete theme structure (e.g., `theme.json`, template files, patterns, etc.).
3. **Preview/Download**: Allow the user to download the theme as a valid, runnable zip file or view a visual mock-up of the generated theme structure.

The generated block theme must be visually impressive, not generic looking, well structured, smartly using standard blocks when possible, and ready for safe installation, demonstrating the AI's ability to handle design and structure.

Core Requirements

Your solution must include:

1. **User Input Interface**: Your choice on the best way to get the site description from the user. Think about the best way to understand what the user wants within good UX boundaries.
2. **AI Orchestration**: A clear component responsible for prompt construction, ensuring the model understands the constraints of WordPress Block Themes (e.g., structure, supported blocks).
3. **Structured Output**: The AI must output valid JSON and template files (or the content that forms them) that adhere to the WordPress Block Theme standards.
4. **No Custom HTML Block**: The generated theme content (templates, parts, patterns) must *exclusively* use the native WordPress block syntax (e.g., `<!-- wp:paragraph -->`, `<!-- wp:post-featured-image -->`) and cannot use the HTML block.
5. **Deliverable Theme Package**: The application must package the generated files into a structure that can be zipped and installed as a working WordPress Block Theme.

Technical Constraints

- **Standalone application**: No external services or credentials required from us to run it, outside of the chosen AI provider.
- **Language agnostic**: The generated *output* is WordPress-specific (JSON/PHP/HTML), but the application's *stack* is your choice.
- **Your choice of AI provider**: OpenAI, Anthropic, local models — your call. The integration must be clean enough that swapping providers is not a major rewrite.
- **Your choice of stack**: Use the stack you are most effective with. The quality of the generated theme is more important than the choice of framework.

Quality Bar

This is an MVP, but it should be a production-minded one. We are looking for code that reflects good engineering discipline, focusing on reliable data transformation and robust output generation.

Error Handling

- Graceful handling of API failures and rate limits.
- Robust validation of the AI's output: if the model generates invalid JSON or malformed block HTML, the application must detect this and provide a meaningful error to the user, not a silent failure.
- Input validation where it matters (e.g., ensuring theme slug is valid).

Testing

- Unit tests covering the core logic (prompt construction, output validation, file packaging).
- At least one integration test covering the end-to-end flow of generating a theme from a simple description.
- Tests should pass cleanly — a simple `npm test` or similar command should run the checks without setup friction.

Git Hygiene

- Incremental, well-scoped commits with clear subject lines.
- Pull requests that are small and focused, even when working solo.
- A commit history that tells the story of how you built this.

Expected Deliverables

1. Working Application

A functional prototype demonstrating the core workflow: input criteria, generate theme, and download the resulting zip file. It should run with minimal setup on a reviewer's machine.

2. README

- How to run it locally, including any environment variables needed (e.g., for the AI API).
- A brief overview of the architecture, especially the output generation process.
- Known limitations or rough edges.

3. Architectural Decision Record (ADR)

A short document covering:

- The key technical decisions made (e.g., how you guided the AI to be reliable yet creative, why you chose a specific AI model).
- Alternatives considered (e.g., different output formats) and why they were rejected.
- Trade-offs accepted consciously (e.g., scope limitations on supported blocks).
- Security considerations (e.g., handling user-provided strings in theme data).
- Design exploration (e.g. how did you get the AI to produce creative, unique designs).

4. What I'd Do Next

A brief section outlining:

- What you'd prioritize if you had another week on this.
- What you'd need to change to make this genuinely production-ready (e.g., a formal block validation pipeline).
- How you'd approach scaling this to support complex dynamic features.

5. Code

- Clean, typed, tested, and linted.
- PRs and commits that give a reviewer confidence in how you work.

Suggested Approach

Phase 1 (4-6 hours): Foundation & Output Schema

Phase 2 (6-8 hours): AI Prompt Engineering & Integration

- Choose your AI provider.
- Develop the initial prompt, focusing on forcing the model to generate structured JSON that describes the theme's templates and patterns using **valid block HTML/comments**.
- Implement the AI call and the initial mapping of AI output to the target file structure.

Phase 3 (6-8 hours): Theme Assembly

- Implement the logic to dynamically create template files, pattern files, and the `theme.json`.
- Implement the file packaging logic (creating the ZIP archive).
- Add visual structure to the UI for inputting criteria.

Phase 4 (4-6 hours): Quality Pass

- Write and pass your tests, especially testing the output validator against expected errors.
- Handle edge cases (e.g., very long descriptions, complex block nesting).
- Ensure the generated theme installs and activates without warnings in a vanilla WordPress environment.

Phase 5 (2-4 hours): Documentation

- README, ADR, and "What I'd Do Next."
- Final review of commits and PR structure.

Bonus Ideas

These are not required, but good targets to go after if you have time. Prioritize everything else up to this point first.

- **Live Theme Preview**: Provide a way for the user to see the resulting theme without needing a separate WordPress instance.
- **Iteration**: Give the user a way to make subsequent changes to the theme. Ideally this comes after a live preview.
- **Pattern Library Integration**: Structure the output so that generated patterns are immediately available in the WordPress Pattern Library interface.
- **Your own ideas:** Be creative! Come up with your own neat and valuable ideas.

Evaluation Criteria

Must have:

- A theme generation process that creates a valid, runnable WordPress Block Theme.
- **Zero usage of the Custom HTML block** for structure or content.
- AI integration that provides structured output (JSON/Block Markup) reliably.
- Robust validation of the AI's output structure.
- Tests that pass cleanly.
- Clean, readable, well-structured code.
- Strong commit history and PR discipline.
- README, ADR, and "What I'd Do Next."

Raises the bar:

- A generated theme that is visually high-quality, non-generic, and demonstrates sophisticated block usage (e.g., Query Loop patterns, advanced layout techniques).
- Thoughtful prompt engineering that produces consistently correct and detailed block markup.
- An ADR that shows genuine product and architectural thinking regarding structured data generation.
- A "What I'd Do Next" that addresses the unique challenges of building a dynamic file generation tool.

A Note on AI Tools

You're building an AI-powered product — it would be strange not to use AI to help build it. Use whatever tools make you most effective. That said, AI-generated code needs to meet the same quality bar as anything else in this codebase. The output of the AI (the theme) and the code you write to manage it (the app) must both be high quality.